

Laboratoire 4

SIMD

Départements : TIC
Unité d'enseignement HPC

Auteur : Urs Behrmann
Professeur : Alberto Dassatti
Assistant : Bruno Da Rocha Carvalho
Classe : HPC
Salle de labo : A09
Date : 21.04.2026

Table des matières

1	Objectif	3
2	Ce que j'ai fait	3
3	Observations simples	3
4	Optimisation du code (Partie 3.4)	4
4.1	3.4.1 Aggraver le cas	4
4.2	3.4.2 Passage en SoA	4
4.3	3.4.3 Version hybride alignée	4
4.4	Conclusion sur 3.4	4
5	Partie 4 – Optimisation manuelle	5

1 Objectif

Le but du laboratoire est de comprendre comment le SIMD peut accélérer un programme, d'abord avec l'auto-vectorisation du compilateur, puis avec une optimisation manuelle.

2 Ce que j'ai fait

J'ai travaillé sur la partie auto-vectorisation avec `simd_demo`.
J'ai compilé et comparé plusieurs options:

Option	Temps d'exécution
-O0	252 ms
-O2	40 ms
-O3	37 ms
-O3 -march=native	27 ms

J'ai aussi utilisé `perf record` et `perf annotate` pour regarder l'assembleur.

3 Observations simples

- En -O0, le code est surtout scalaire.
- En -O2, on voit des registres `xmm` et des instructions packed (ex: `mulps`, `addps`), donc la boucle est vectorisée.
- En -O3, c'est proche de -O2 sur ce code, donc petit gain seulement.
- Avec `-march=native`, le compilateur utilise des registres `ymm` (AVX/AVX2), donc plus de données traitées en parallèle.

En résumé, le plus grand gain vient du passage de -O0 à -O2. Ensuite -O3 apporte un petit plus, et `-march=native` apporte encore un gain grâce aux instructions du processeur local.

4 Optimisation du code (Partie 3.4)

4.1 3.4.1 Aggraver le cas

J'ai modifié la structure pour ajouter des données inutiles entre les informations utiles. Dans l'assembleur (`main_opti_1.txt`), on voit surtout des opérations scalaires (`...ss`) et presque pas de vectorisation utile sur la boucle principale.

Ce résultat est logique : les valeurs `position` et `velocity` sont plus éloignées en mémoire, donc le compilateur ne peut pas charger facilement des blocs de données homogènes.

4.2 3.4.2 Passage en SoA

J'ai transformé le code en Structure of Arrays, avec deux tableaux séparés : un pour `position` et un pour `velocity`.

Dans `main_opti_2.txt`, on voit cette fois de vraies instructions vectorielles sur plusieurs éléments (`yvmadd...ps`, `vcmpltps`, etc.). Le code est plus favorable au SIMD car les données du même type sont contiguës en mémoire.

4.3 3.4.3 Version hybride alignée

J'ai ensuite fait une version hybride par blocs :

- `BLOCK_SIZE = 16`
- `ALIGNMENT = 64`
- allocation avec `posix_memalign`

Dans `main_opti_3.txt`, on observe bien la logique par blocs et des instructions AVX vectorielles. Le compilateur traite les blocs de manière régulière, puis gère les éléments restants avec une partie scalaire.

4.4 Conclusion sur 3.4

Sur mes tests, l'organisation mémoire a un impact direct sur la vectorisation. Pour le compilateur, il est difficile de faire de la vectorisation s'il y a des données non contiguës en mémoire. Il faut avoir des 'vecteurs' de données contiguës pour que le compilateur puisse détecter qu'il peut faire de la vectorisation.

5 Partie 4 – Optimisation manuelle

J'ai testé trois stratégies SIMD :

- AoS + gather AVX2
- SoA persistant + chargements contigus
- version hybride alignée par blocs

Les trois versions donnent les mêmes résultats que la version séquentielle.

La différence entre les trois versions vient surtout de l'organisation des données et du coût d'accès mémoire :

- En AoS + gather, les données sont entremêlées, donc on doit faire des lectures indirectes (gather), ce qui coûte plus cher.
- En SoA persistant, les tableaux **x**, **y**, **z**, **bias** sont contigus. On peut charger 8 entiers d'un coup avec des lectures simples, donc la boucle SIMD est plus efficace.
- En hybride, même si les blocs sont alignés, il faut d'abord réorganiser les données en blocs avant de calculer. Ce coût de préparation annule le gain attendu sur ce cas.

Mesures faites avec **n** = 30 000 000 éléments :

Version	Temps
Séquentiel	122–123 ms
AoS + gather AVX2	161 ms
SoA persistant (garde)	112–113 ms
Hybride aligné par blocs	355 ms

Par contre, en performance, la version SoA persistante est la meilleure dans mon cas. La version hybride a bien été testée, mais elle n'a pas apporté de gain : elle est même plus lente, à cause du coût de réorganisation en blocs et des accès supplémentaires.

On observe environ 10 ms de gain entre la version séquentielle et la version SoA persistante, ce qui correspond à un gain d'environ 8% sur ce test.

J'ai donc gardé uniquement la version SoA persistante dans le code final.